

```
[28]: similar_words = model.wv.most_similar('machine', topn=3)
      print(similar_words)

[('gensim', 0.38692590594291687), ('way', 0.1673559993505478), ('vec', 0.1358923763036728)]
```

Figure 4: Getting words similar to a given word

```
: vector = pretrained_model['learning']
  print(vector)

[-0.08837891  0.1484375 -0.06298828  0.11767578  0.10986328  0.04345703
  0.05493164  0.04394531 -0.06152344 -0.05932617  0.20125  0.00105469
 -0.16113201 -0.06396484  0.04516602 -0.06005059 -0.10066406  0.20507012
  0.08105469 -0.16601562 -0.02526855  0.17578125 -0.00171661  0.21386719
 -0.01196289 -0.16894531  0.02270508  0.09277344 -0.02685547 -0.09423828
 -0.06542969  0.0703125 -0.0703125 -0.04907227 -0.0246582 -0.17089844
  0.00848389 -0.07128906  0.09472656 -0.17871094  0.07128906  0.05297852
 -0.01599121 -0.0625 -0.05273438 -0.09082031  0.11101641  0.22460938
 -0.17382812  0.17089844 -0.00469971  0.13769531 -0.10693359  0.02478027
  0.18554688  0.40625 -0.03491211 -0.18261719  0.09912109 -0.17285156
  0.03039551  0.06787109  0.09375 -0.29882812  0.00866699  0.09130859
 -0.16308594 -0.05761719  0.08349609 -0.05371094 -0.06103516 -0.04125977
  0.01506914 -0.04370117 -0.00671387  0.04956055  0.16308594  0.06396484
  0.15234375  0.14941406  0.11572266 -0.04272461  0.06225506  0.00806719
  0.06689453 -0.19433594 -0.08447266  0.21386719  0.05419922  0.078125
  0.0625 -0.03686523 -0.13476562 -0.26953125 -0.01778002  0.02185059
  0.23144531 -0.13769531  0.09765625 -0.09716797 -0.07080078 -0.125
  0.00921631  0.09423020 -0.00370410  0.00933030  0.14941406 -0.03979492
  0.17089844 -0.30078125 -0.31054680  0.29082012 -0.14257812 -0.16992100
 -0.01281738  0.16992100  0.05322266  0.01391602  0.0612793 -0.0480957
 -0.23144531  0.11328125  0.04077148  0.14648438 -0.13769531 -0.10986328
 -0.11669922  0.1015625 -0.3046875  0.35351562  0.05517578 -0.19140625
 -0.37695312 -0.06445312  0.03808594 -0.09228516  0.15429688 -0.08251953
 -0.11425701  0.10457031 -0.19140625 -0.02605547 -0.24902344 -0.15429688
 -0.03540039 -0.10449219  0.10595703  0.05126953  0.06347656 -0.15917969
  0.19628906  0.05371094 -0.33007812  0.04589844 -0.04614258  0.07910156
 -0.08300781 -0.20996094 -0.2265625 -0.05004883  0.13085938  0.33007812
 -0.04736328  0.20703125 -0.13378906  0.03955078  0.28320312 -0.00613403
  0.11376953 -0.02209473 -0.10693359  0.01190106 -0.14257812 -0.04931641
  0.21209062  0.16503906  0.00430290 -0.0059375  0.05395500  0.1171075
  0.07226562  0.03613281  0.44726562  0.21972656  0.0055542  0.18945312
 -0.234375 -0.09521484  0.21582031 -0.25976562 -0.07470703 -0.2109375
  0.00543213  0.10791016  0.02844238 -0.10839844  0.12207031 -0.24804688
 -0.05883789 -0.20019531 -0.07958984 -0.2109375  0.13085938 -0.2734375
 -0.21191406 -0.09423020 -0.12109375 -0.09063201 -0.06176750 -0.15722656
  0.04410945  0.11132012  0.03033000 -0.21502031  0.00390430  0.05901445
  0.17089844 -0.03125 -0.16601562  0.19335938  0.13085938 -0.16113281
  0.05932617  0.01501465  0.06079102  0.02148438  0.05053711 -0.12304688
 -0.00534058 -0.11621094  0.11279297 -0.09130859  0.15039062  0.01623535
  0.27539062 -0.10457031 -0.07421875  0.02929688 -0.0213623  0.12150203
 -0.05004803  0.05615234 -0.00174713 -0.13671075 -0.17675701 -0.19620906
 -0.03076172  0.12695312  0.03881836 -0.28125  0.03491211  0.12304688
  0.02819824  0.29492188 -0.04663086 -0.25585938 -0.19726562  0.05151367
 -0.16210938  0.03466797  0.01989746  0.10986328 -0.07910156  0.06591797
 -0.11181641  0.07519531  0.44140625 -0.06005859 -0.328125  0.28125]
```

Figure 5: Using the GoogleNews Word2Vec model

```
pretrained_model.most_similar(positive=['king', 'woman'], negative=['man'], topn=1)

[('queen', 0.7118193507194519)]
```

Figure 6: Vector embedding of the word 'Queen'

Now let us look at one of the most popular Word2Vec models, which is the 'GoogleNews-vectors-negative300.bin' model. This is a model that has been trained on a huge dataset. It was trained on the Google News corpus, which

most words we use. You can download it from <https://huggingface.co/Nathan1111/word2vec-google-news-negative-300-bin>.

Once downloaded, it can be used for a wide variety of tasks such as text classification, machine translation, and more. Let us see how we can use this model to get the vector representation of the word 'learning' as shown in Figure 5.

from gensim.models import KeyedVectors

```
model_path = "/home/jmittap/Downloads/
GoogleNews-vectors-negative300.bin"
pretrained_model = KeyedVectors.
load_word2vec_format(model_path,
binary=True)
```

```
vector = pretrained_model['learning']
print(vector)
```

This model is proof that Word2Vec models capture the meaning in text. The most popular example used for showing this is that the following:

→ The $\text{vec}(\text{King}) - \text{vec}(\text{Man}) + \text{vec}(\text{Women})$

...gives the vector embedding of the word Queen! How cool is that! This can be done as shown in Figure 6. This proves that the meaning of things like 'Royalty' has been captured by this model!

You can now try and use this word embedding technique in different NLP and ML projects! 

contains billions of words. This model can give vector representations of

By: Jishnu Saurav Mittapalli

The author is a data scientist interested in modern technologies like artificial intelligence and machine learning. He is currently working in the field of natural language processing.